DAMG 6300 · GEN AI WITH LLMS IN DATA ENGINEERING · FALL 2026

Vroomcheck

Data model, data architecture and system architecture

Rudrang Gade · Ethan Gomes · Akshit Verma · 24 October 2026 · data snapshot: NHTSA files of 21 October 2026

> **Business question.** Can we detect emerging vehicle safety defects from public owner complaints early enough to act before a recall, with every alert traceable to evidence?
> Statistics detect, AI explains, and a verifier checks every claim.

Part 1 · Data model	the star schema: facts, bridges, dimensions, rules, tests
Part 2 · Data architecture	how data flows from NHTSA into DuckDB and Snowflake
Part 3 · System architecture	the components and how they connect (short)
Speaker notes	two one-page scripts for a 6–7 minute talk

Competency questions the system answers

1. **Emerging defects:** which complaint patterns are rising abnormally with no recall yet?
2. **VIN check:** for a given vehicle, which recalls apply, and what do owners report that no recall covers?
3. **Similar past recalls:** which earlier recalls, from any manufacturer, look like this pattern?
4. **Back-test:** how early would past recalls have been flagged?

Success targets and current results

Target	Goal	Now
Past recalls flagged before the recall date (of detectable)	≥ 50%	**34.8%** (153 of 440), below target
Median early-warning lead time	≥ 3 months	**9.0 months**, met
Lead over NHTSA opening an investigation (median)	(no target)	**11.1 months**
Brief claims that cite a real record ID	100%	**100%**, enforced by the verifier

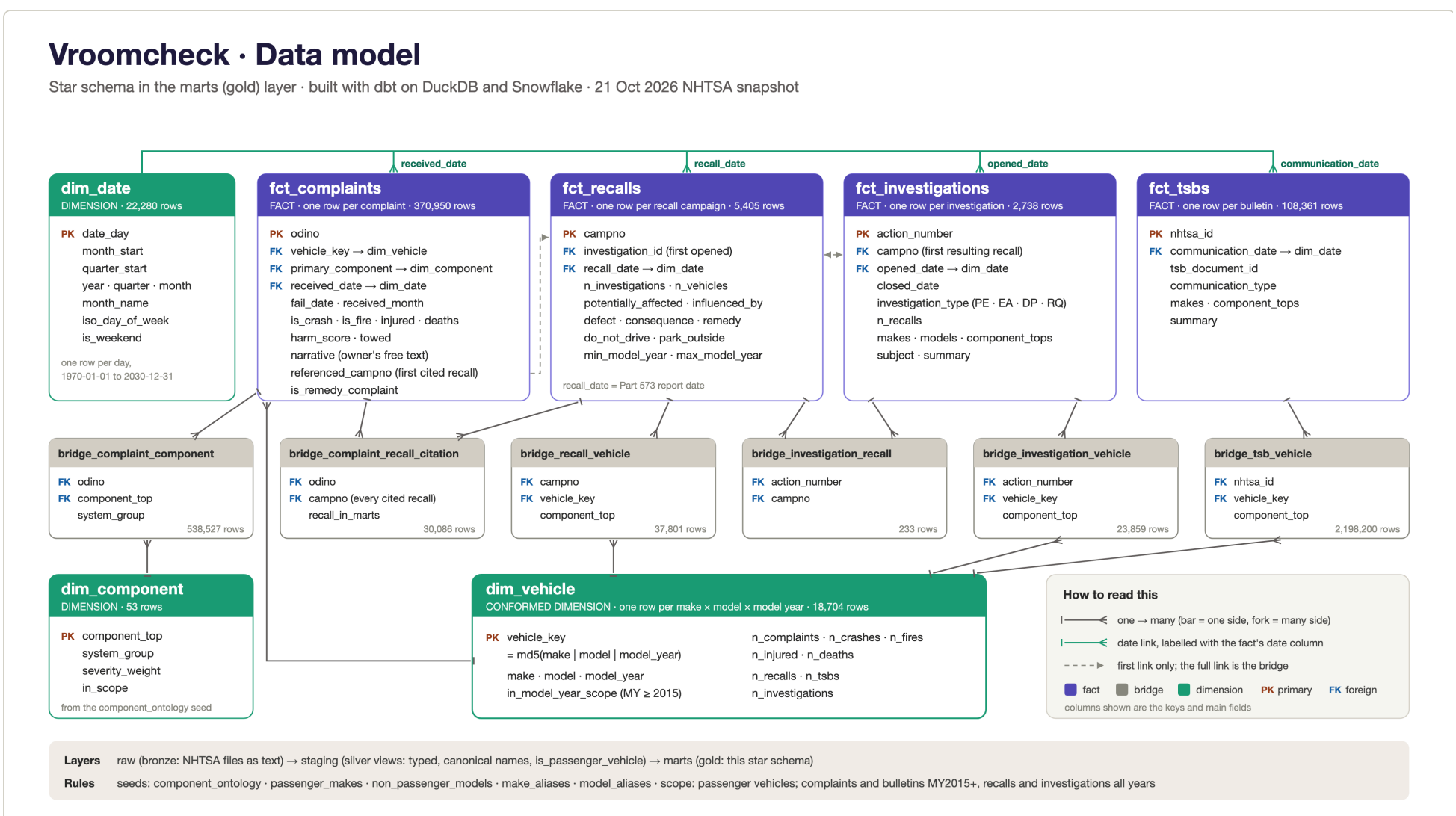


Figure 1 · The gold-layer star schema: four fact tables, six bridge tables and three dimensions. Green lines are date links; dashed arrows are "first link" shortcut columns.

PART 1

Data model

> The data model turns four messy NHTSA bulk files into one star schema: four fact tables for the four kinds of events, joined through bridge tables to a shared vehicle dimension, plus component and date dimensions, so complaints, recalls, investigations and bulletins can all be compared for the same vehicle.

1. Why the project needs a data model

The raw NHTSA files cannot be analysed directly, for four reasons. These explain every design choice that follows.

1. **Rows are repeated (fan-out).** NHTSA writes one row per event per component per vehicle. A complaint that names three components appears three times; a recall covering 40 vehicles appears 40+ times. Counting raw rows overcounts events.
2. **Names do not match.** The same car is spelled differently across files (`GLA250` / `GLA 250`, `F150` / `F-150`, `MERCEDES BENZ` / `MERCEDES-BENZ`), so joins silently miss.
3. **Mixed vehicle types.** "Vehicle" recalls include RVs, school buses, fire trucks, motorcycles and trailers. The project is about passenger cars.
4. **No direct links.** Complaints do not say which recall fixed them; investigations and recalls are linked only by a campaign number.

2. The layers

Layer	What it holds	Why
Raw (bronze)	NHTSA files exactly as published, every column as text, plus source file and load time	Nothing is lost or changed; every value traces back to the source
Staging (silver)	The same rows, cleaned: real dates and numbers, canonical make/model names, a passenger-vehicle flag	One place where all cleaning rules live
Marts (gold)	The star schema in Figure 1, one row per real event	What analysis, detection and the dashboard use
Ref (seeds)	Business rules as small CSV tables	Changing scope means editing a CSV, not code

The same dbt project builds all of this in both **DuckDB** and **Snowflake**, and the two produce identical tables.

3. The four fact tables

Each fact has one row per real-world event, so counting rows counts events.

fct_complaints: the early-warning signal (370,950 rows)

- **Grain:** one owner complaint, key `odino`. 1.1 million raw rows collapse to one per complaint.
- **Scope:** passenger vehicles, model year 2015 or later, received 2015 or later, narrative of at least 20 characters. Tire, child-seat and equipment-only complaints are removed.

- `narrative`: the owner's own words, which feed the AI detection.
- `primary_component`: the most severe component the owner named; ties are broken alphabetically so results are reproducible.
- `harm_score` = 1 + 3·crash + 3·fire + 2·injury + 5·death + 1·towed; ranks how dangerous a pattern is.
- `referenced_campno`: a recall number the owner typed into the text, a direct complaint → recall link.
- `is_remedy_complaint`: the call-centre "waiting for recall parts" template, about delays rather than new failures, so detection excludes it.

fct_recalls: the ground truth (5,405 campaigns)

- **Grain:** one recall campaign, key `campno` (e.g. `24V123000`).
- `recall_date` is the date the manufacturer filed its Part 573 defect report with NHTSA: the date early warning is measured against.
- Also the defect, consequence and remedy text, units affected, who initiated it (manufacturer or NHTSA), and the do-not-drive and park-outside advisories.

fct_investigations: NHTSA's own timeline (2,738)

- **Grain:** one NHTSA investigation, key `action_number`. The prefix is the stage: PE preliminary evaluation, EA engineering analysis, DP defect petition, RQ recall query.
- `opened_date` shows we are earlier than the regulator: about 11 months before NHTSA opens an investigation, at the median.

fct_tsbs: manufacturer bulletins (108,361)

- **Grain:** one service bulletin, a manufacturer telling dealers how to fix a problem.
- Supporting evidence: a bulletin often shows the manufacturer knew about an issue before any recall. The AI briefs cite these.

The **dashed arrows** between facts are convenience shortcuts: "the first recall this complaint cites" and "the first investigation behind this recall". The complete many-to-many links live in the bridges.

4. The six bridge tables

A bridge exists wherever one thing relates to many of another in both directions. Instead of repeating rows in a fact, each pair is stored once.

Bridge	Connects	Why it is needed	Rows
`bridge_complaint_component`	complaint ↔ component	32% of complaints name more than one component	538,527
`bridge_complaint_recall_citation`	complaint ↔ recall it cites	some owners cite several recalls; all are kept	30,086
`bridge_recall_vehicle`	recall ↔ vehicle	one recall covers many make / model / years	37,801
`bridge_investigation_recall`	investigation ↔ recall	one probe can lead to several recalls and vice versa	233
`bridge_investigation_vehicle`	investigation ↔ vehicle	one probe covers many vehicles	23,859

`bridge_tsb_vehicle`	bulletin ↔ vehicle	one bulletin covers many vehicles	2,198,200

5. The three dimensions

dim_vehicle: the hub of the model (18,704 rows)

- **Grain:** one make × model × model year, e.g. "2021 Jeep Wrangler".
- **Key:** `vehicle_key = md5(make | model | model_year)`, computed the same way in every source after name cleaning. This is what lets all four sources line up for the same car.
- **Conformed:** it contains every vehicle referenced anywhere in the marts, and every link to it is tested.
- `in_model_year_scope` marks MY2015+. Scope is an attribute, not a filter, because the VIN check and the investigation history need older vehicles.
- Pre-computed counts per vehicle: complaints, crashes, fires, injuries, deaths, recalls, bulletins, investigations.

dim_component (53 rows)

- One row per NHTSA component with its **system group**, a **severity weight** from 0 to 1 and an in-scope flag. The weight decides each complaint's primary component and feeds alert priority, so an air-bag cluster outranks an infotainment one.

dim_date (22,280 rows)

- One row per day, 1970–2030, with month, quarter, year, weekday and weekend flag. All four facts link to it: complaints by `received_date`, recalls by `recall_date`, investigations by `opened_date`, bulletins by `communication_date`. The project is about timing, so one shared calendar keeps every monthly and yearly comparison consistent.

6. The rules that shape the data

- **Passenger vehicles only, one rule everywhere:** an allow-list of 75 passenger brands (current and defunct, such as Pontiac and Saturn), minus 85 model patterns for motorcycles, medium-duty trucks and motorhome chassis those brands also sell (Honda Gold Wing, BMW R 1250 GS, Ford F-650).
- **Name aliases:** 37 model spellings and 1 make spelling map to one canonical name; NHTSA's "REDUNDANT" prefix is removed.
- **Model years:** complaints and bulletins MY2015+; recalls and investigations keep all years.
- **Implausible harm values:** counts above 20 injured or 10 deaths (owners typing 99 as a protest) are treated as unknown and flagged.

7. How we know it is correct

- **64 automated dbt tests** on every build: unique and non-null keys, every relationship (each bridge to its fact and to `dim_vehicle`, fact dates to `dim_date`), harm limits and consistency checks.
- **4 warnings** flag source-data oddities (e.g. 7 complaints dated before their failure, 2 investigations that close before they open). They are reported, not hidden.
- **Reproducible:** two rebuilds give identical output.
- **Same on both platforms:** the Snowflake build is identical to DuckDB, table by table and row by row.

8. How the model serves the project's questions

Question	Tables that answer it
Which patterns are emerging with no recall?	`fct_complaints` (narratives per vehicle line); `bridge_recall_vehicle` + `fct_recalls` (already recalled?)
For this VIN, which recalls apply?	`dim_vehicle` → `bridge_recall_vehicle` → `fct_recalls`; matches NHTSA's live API (15 of 15 for a 2021 Wrangler)
Which past recalls look like this pattern?	`fct_recalls` defect text, matched by meaning
Would we have flagged past recalls early?	`fct_recalls.recall_date` and `fct_investigations.opened_date` versus complaint dates, via `dim_date`

9. Design decisions

- **Star schema with bridges, not one wide table:** each fact keeps its true grain, so counts are honest, and many-to-many links do not duplicate data.
- **One conformed vehicle key:** the single most important decision; it makes cross-source comparison possible.
- **Rules as data (seeds):** scope changes without code changes, and reviewers can read the rules directly.
- **Raw kept untouched:** a full audit trail back to NHTSA.

Likely questions

Why not one big table? The sources have different grains; joining them flat multiplies rows (one recall × 40 vehicles × 3 components).

Why is dim_vehicle larger than the vehicles with complaints? It also holds vehicles seen only in recalls, investigations or bulletins, including older model years: it is the shared list for everything.

What is the dashed arrow? A shortcut column holding the first link; the full link is in the bridge.

Why did recalls drop from 13,701 to 5,405? Most "vehicle" recalls NHTSA publishes are for RVs, buses, trailers and commercial trucks. They are out of scope, and no in-scope complaint could ever match them.

How do you handle NHTSA changing its data? Every run rebuilds from the latest files; keeping history with dbt snapshots is a planned next step.

Where is the AI? In a separate ML layer on top of this model (embeddings, clusters, alerts). The model makes sure the AI works on clean, correctly linked, in-scope data.

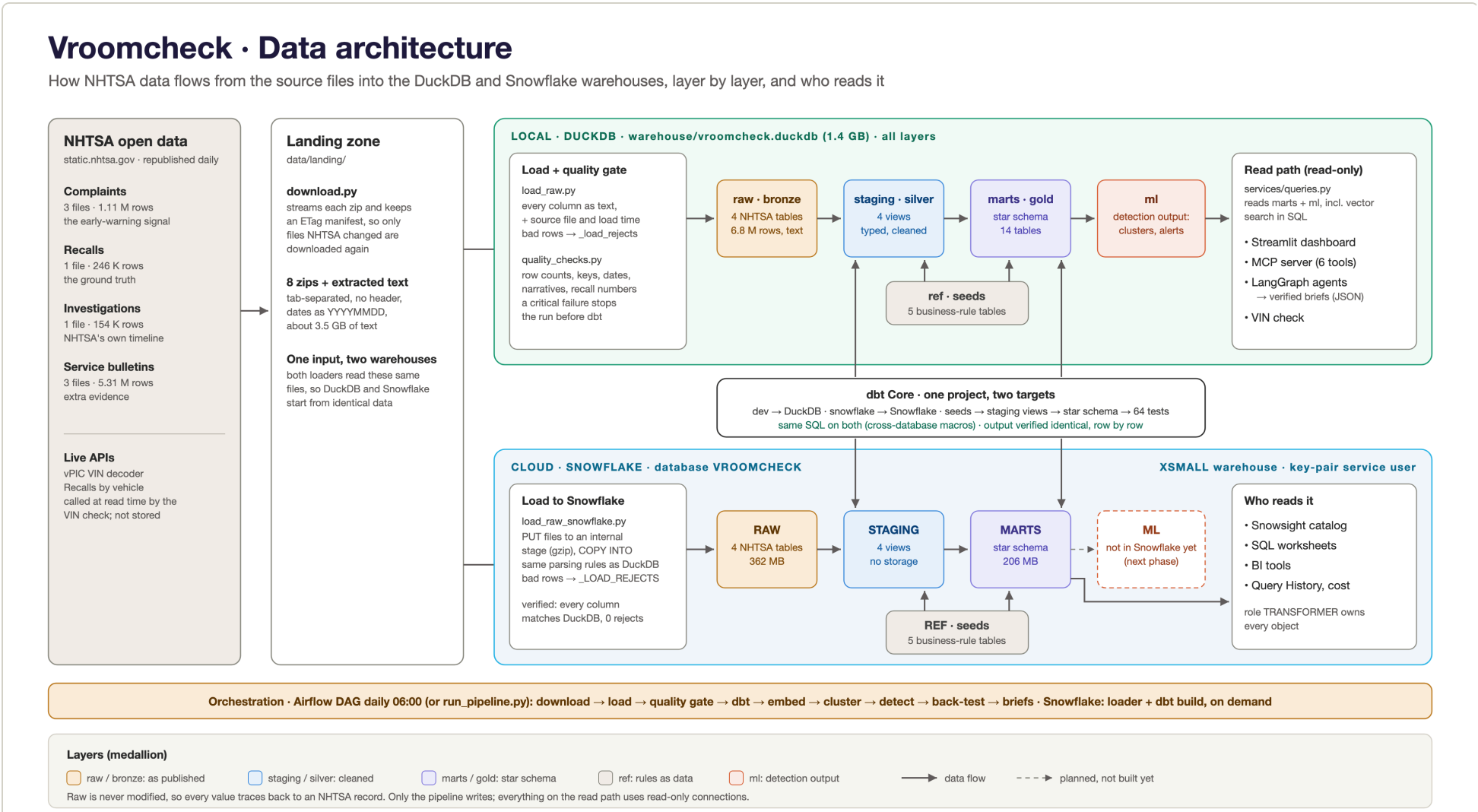


Figure 2 · Data architecture: one landing zone feeds two warehouses. DuckDB holds every layer including the ML output; Snowflake holds raw → marts, built by the same dbt project.

PART 2

Data architecture

> Vroomcheck is a daily batch pipeline that pulls NHTSA's public safety data into a layered warehouse, finds unusual complaint patterns with machine learning, and serves them through AI-written, fact-checked briefs, a dashboard and an API that AI assistants can call.

1. Two paths around one warehouse

- **Write path (daily batch):** source → ingest → transform → detect. Only the pipeline writes.
- **Read path (on demand):** warehouse → read-only query service → agents, dashboard, MCP server. It only reads.

Keeping these separate means users and AI tools can query at any time without interfering with the nightly build, and the pipeline can rebuild everything from scratch safely.

2. Sources and landing zone

- **NHTSA bulk files** (static.nhtsa.gov, republished daily): complaints, recalls, investigations and service bulletins, 8 zip files and about 6.8 million raw rows.
- **Live NHTSA APIs** (vPIC VIN decoder, Recalls by vehicle) are called only at read time, for the VIN check.
- All data is public; no personal data is collected.
- **Landing zone** (`data/landing/`): `download.py` streams each zip and keeps an ETag manifest, so a daily run re-downloads only the files NHTSA changed. Both warehouses load from these same files, so they start from identical data.

3. Write path

Step 1 · Ingest (Python)

Part	What it does	Why
`load_raw.py`	Loads the tab-separated files into `raw.*` with every column as text, exactly as published, plus lineage columns. Unreadable rows go to `raw._load_rejects` instead of being dropped	Nothing is lost or silently altered
`quality_checks.py`	The data-quality gate: row counts, missing keys, unparseable dates, rejected rows, missing narratives, future dates, recall-number format. Results go to `raw._dq_results`	A critical failure stops the pipeline before dbt runs, so bad data never reaches analysis
`load_raw_snowflake.py`	Uploads the same files to a Snowflake internal stage (gzip) and runs `COPY INTO RAW.*`, with the same parsing rules; bad rows go to `RAW._LOAD_REJECTS`	Verified identical to DuckDB on every column, with 0 rejects

Step 2 · Transform (dbt Core)

- **Seeds:** the business rules (component ontology, passenger-vehicle rules, name aliases).
- **Staging views:** type and clean the raw rows.

- **Marts:** the star schema (4 facts, 6 bridges, 3 dimensions).
- **Tests:** 64 automated checks on keys, relationships and data quality.
- **One project, two targets:** `dev` builds DuckDB and `snowflake` builds Snowflake. Cross-database macros keep the SQL identical, and the outputs match row for row.

Step 3 · Detect (Python ML)

1. **Embeddings:** strip call-centre boilerplate ("THE CONTACT OWNS A…"), then turn each narrative into a 384-number vector with the `all-MiniLM-L6-v2` sentence model, so complaints that mean the same thing get similar vectors. Incremental: only new complaints, for vehicle lines with at least 150 complaints.
2. **Clustering:** one run per vehicle line (make + model, model years pooled): reduce to 32 dimensions, group with HDBSCAN, which finds groups without being told how many and leaves one-off complaints as noise. Each cluster gets keyword labels.
3. **Spike test:** for every cluster and month, compare the last 3 months with the 12 months before, adjusted for fleet growth, and use a Poisson test to ask whether the rise is too large to be chance. It is causal: each month uses only data available at that time.
4. **Recall matching:** link a cluster to a recall when owners cite its campaign number (at least 20% of the cluster) or when the cluster's meaning matches the recall's defect text, with bonuses for component and timing agreement.
5. **Alerts:** EMERGING (spiking recently, no matching recall), RECALLED, WATCH or NON-DEFECT, ranked by priority = volume × spike strength × harm × component severity.
6. **Back-test:** for each past recall since 2017, would we have flagged it before the recall date? (Section 6.)

4. Warehouse and platforms

- **Local:** one DuckDB file (about 1.4 GB) with five schemas: `raw`, `staging`, `ref`, `marts` and `ml`.
- **Cloud:** Snowflake database `VROOMCHECK` with `RAW`, `REF`, `STAGING` and `MARTS` (about 570 MB). The ML layer stays local for now, by design.
- **Snowflake security and cost:** a dedicated service user with key-pair login, a least-privilege role (`TRANSFORMER`), and an XSMALL warehouse that suspends after 60 seconds idle. The full load and build used well under one credit.
- **Vectors in the warehouse:** embeddings are stored as fixed-length arrays, so similarity search runs in plain SQL (`array_cosine_similarity`), with no separate vector database.
- **One writer at a time:** DuckDB allows a single writer, so the steps run in sequence; on Snowflake they could run in parallel.

5. Read path

- **Query service** (`services/queries.py`): one shared layer with read-only connections for alerts, cluster details, representative complaints, vector search over past recalls, related bulletins and investigations, the VIN check and the back-test summary. The dashboard, MCP server and agents all use it, so they always agree.
- **LangGraph agents:** signal → retrieval (RAG) → drafting (LLM) → verifier write a cited brief per emerging cluster (see Part 3).
- **Streamlit dashboard** (5 views), **MCP server** (6 tools for Claude and other assistants), and the **VIN check** (vPIC decode plus a live recall cross-check).
- **In Snowflake:** Snowsight catalog, SQL worksheets, BI tools, and Query History for cost and lineage of what ran.

6. How the early-warning rate is measured (back-test)

For each past recall, the back-test asks one question: did a complaint cluster for this defect spike *before* the recall was filed?

Stage	Recalls	Rule
In scope	3,230	Passenger-vehicle recalls filed since 1 Jan 2017, model year 2015+, up to the last scored month
Has a matching cluster	622	A cluster on the same vehicle line is linked to the recall (cited ≥ 20%, or semantic score ≥ 0.65)
Detectable	**440**	That cluster had at least 5 complaints before the recall month
Flagged before the recall	**153**	The spike test fired in one of the 24 months before the recall month

- **Spike rule:** at least 4 complaints in the last 3 months, at least 2× the expected count, Poisson chance below 1%, and a 14-month warm-up before any verdict.
- **Result:** 153 ÷ 440 = 34.8% (target 50%). Lead time runs from the end of the alert month to the recall date: median 9.0 months, and 11.1 months ahead of NHTSA opening an investigation.
- **Example:** recall 26V560000 (Ram 1500 radio software, filed 1 Sep 2026) matched cluster `RAM__1500__001`; 205 earlier complaints; first spike December 2025; about 8 months of warning.
- **Caveats:** "of detectable" is a generous denominator (153 ÷ 3,230 ≈ 5% of all in-scope recalls; many recalls have no complaint signal at all). Clusters are built on the full history, and cited matches use post-recall citations, so the cluster-to-recall link uses some hindsight; the alert date itself is strictly causal.
- **Planned improvements:** re-cluster on rolling time windows (removing the hindsight), better recall matching, and normalising by the number of vehicles affected.

7. Orchestration

- **Airflow DAG**, daily at 06:00 (NHTSA refreshes overnight), in Docker: 14 tasks in a fixed order with retries.
- `scripts/run_pipeline.py` runs the same steps locally and can resume from or run a single step.
- Snowflake is refreshed on demand: the Snowflake loader, then `dbt build --target snowflake`.
- Typical full run on a GPU laptop: download ~2 min, raw load ~50 s, dbt ~1.5 min, embed ~10 min (full rebuild), cluster ~4 min, detect and back-test ~1.5 min.

8. One complaint's journey

1. An owner reports that a 2023 Wrangler 4xe lost power in electric mode. NHTSA publishes it; the download picks up the changed file.
2. The raw load stores it as text and the quality gate passes.
3. dbt cleans it, recognises a passenger vehicle, picks its primary component and links it to the 2023 Wrangler in `dim_vehicle`.
4. The embedding places it next to similar complaints, clustering groups them, and the spike test sees the cluster rising far above its baseline.
5. No recall covers it, so the cluster is EMERGING, the top-priority alert.
6. The agents write a brief citing this complaint (`C-…`), a similar Honda recall (`R-…`) and a bulletin (`T-…`); the verifier checks each claim.

7. An analyst sees it in the dashboard, an owner checks their VIN, or Claude asks the MCP server.

9. Design principles

- **Layered and traceable:** raw is never modified; every answer traces back to an NHTSA record.
- **Incremental where expensive, full rebuild where cheap:** downloads and embeddings are incremental; everything else rebuilds each run, which keeps it simple and repeatable.
- **Fail early, fail closed:** the quality gate stops bad loads; the verifier rejects unproven claims.
- **No look-ahead in detection:** the spike test only uses past data, so the back-test is honest.
- **One query layer for every consumer**, and **portable SQL** that runs on DuckDB and Snowflake.

Likely questions

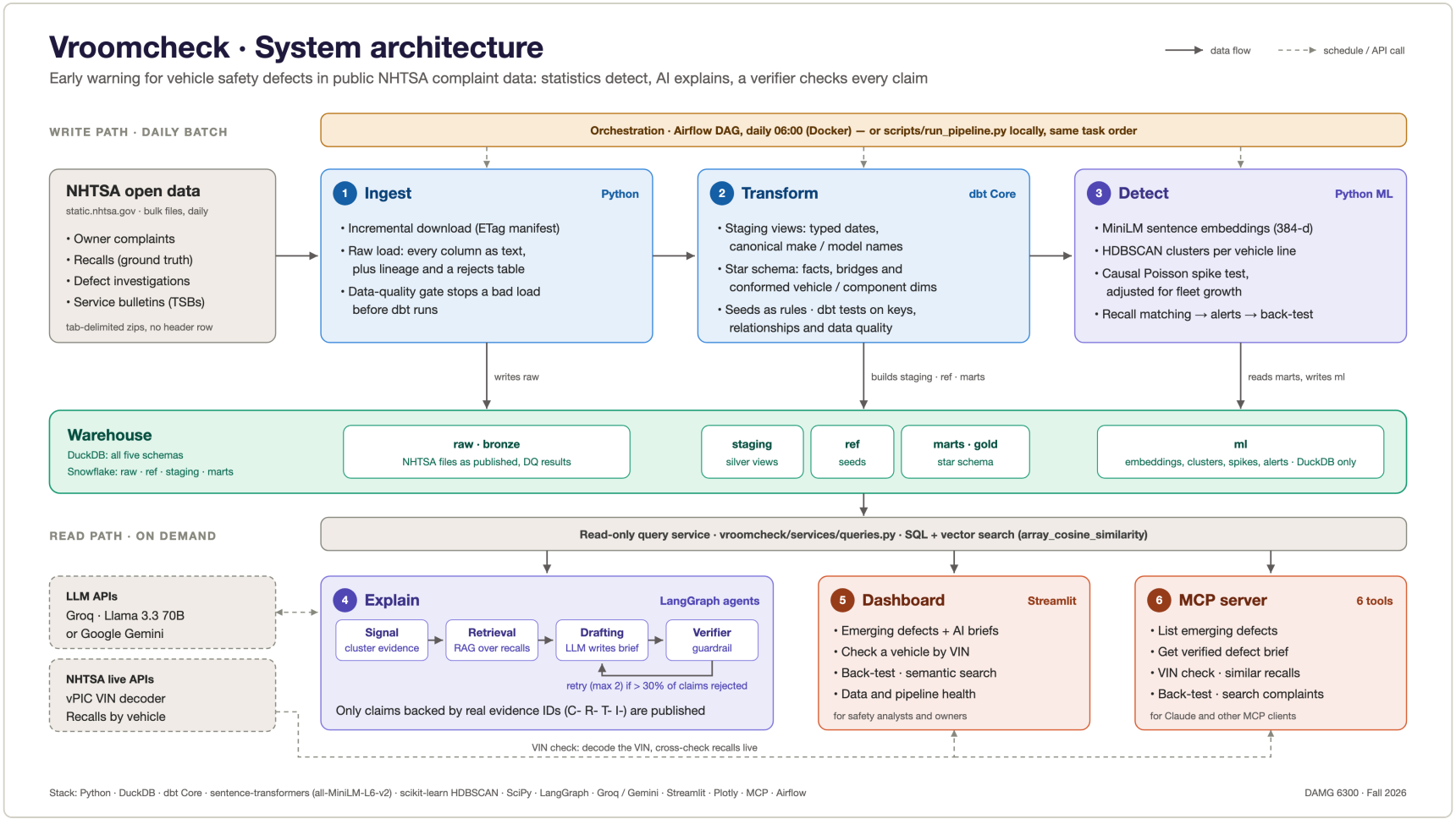
Why DuckDB and Snowflake? DuckDB is free, fast and runs on a laptop for development and demos; Snowflake is the shared cloud warehouse. One dbt project serves both, and we proved they produce identical tables.

Why no separate vector database? At about 350,000 vectors, similarity search runs fine in SQL inside the warehouse: one fewer system to run and keep in sync.

What if the LLM makes something up? Only claims whose citations point to real records, and whose cited text supports the claim, are published; the rest are shown as rejected with the reason.

How do you know detection works? The back-test against past recalls: about 35% of detectable recalls flagged early, a median 9 months before the recall and 11 months before NHTSA opens an investigation.

What next? Schedule Airflow and the Snowflake build, move the ML layer to Snowflake (its VECTOR type and Cortex), keep history with dbt snapshots, and re-cluster on rolling windows.



PART 3 · SYSTEM ARCHITECTURE

The same system, seen as components. The write path (Ingest → Transform → Detect, driven by Airflow) fills the warehouse; the read path serves it through one read-only query service to three consumers.

Explain (LangGraph): signal → retrieval (RAG) → drafting (Llama 3.3 70B via Groq, or Gemini) → verifier. Every claim must cite real `C–` / `R–` / `T–` / `I–` IDs; if over 30% are rejected it redrafts (at most twice). Only verified claims are published.

Deliver: the Streamlit dashboard and the MCP server (6 tools). **External:** LLM APIs for drafting and checking, NHTSA's live APIs for the VIN check. **Motto:** statistics detect, AI explains, a verifier checks every claim.

Data model · about 3 minutes

Total talk 6–7 min: data model ≈ 3:00 · data architecture ≈ 2:30 · system architecture ≈ 0:45 · buffer ≈ 0:30. Have Figure 1 on screen.

Data model 3:00	Data architecture 2:30	System 0:45	+0:30

0:00 OPEN **"Four messy NHTSA files become one star schema, so complaints, recalls, investigations and bulletins can be compared for the same vehicle."**

0:15 WHY Four problems drive the design:

- **Fan-out:** one complaint × 3 components = 3 rows; one recall × 40 vehicles = 40 rows.
- **Names don't match:** GLA250 vs GLA 250, F150 vs F-150.
- **Mixed vehicles:** RVs, buses, motorcycles inside "vehicle" recalls.
- **No links** between complaints, recalls and investigations.

0:45 LAYERS Point at the footer. **Raw** untouched (audit trail) → **staging** cleans → **marts** = this star schema. **Seeds** hold the rules as data. Same dbt project builds **DuckDB and Snowflake, identical row for row**.

1:05 FACTS Top row, one row per real event:

- **Complaints, 371K:** the early signal. Owner's narrative, most severe component, harm score, the recall they cite.
- **Recalls, 5.4K:** ground truth. The Part 573 filing date is what "early" is measured against.
- **Investigations, 2.7K:** NHTSA's own timeline. We're about 11 months ahead.
- **Bulletins, 108K:** manufacturers quietly acknowledging problems. The briefs cite them.

1:50 BRIDGES Middle row. **"Wherever one thing relates to many in both directions, we store each pair once."** One recall → many vehicles; one investigation → several recalls. The largest is bulletins × vehicles, 2.2M.

2:15 DIMS **dim_vehicle is the hub:** one row per make × model × year, key = md5(make | model | year), computed identically for every source, which is the decision that makes comparison possible. **dim_component:** severity weights drive priority (air bags outrank infotainment). **dim_date** (green line): one shared calendar for every fact, because this project is about timing.

2:45 PROOF **64 automated tests** on every build; results are reproducible; Snowflake matches DuckDB row for row. BRIDGE **"So that's what the data looks like. Here's how it gets there."**

If asked. One big table? Different grains; a flat join multiplies rows. · **Dashed arrow?** Shortcut to the first link; the full link is the bridge. · **Recalls 13.7K → 5.4K?** Most NHTSA "vehicle" recalls are RVs, buses, trailers and trucks: out of scope. · **History?** Full rebuild each run; dbt snapshots are next.

Data architecture · about 2½ minutes

Have Figure 2 on screen; switch to Figure 3 (system architecture) for the last 45 seconds.

3:00	OPEN	**"A daily batch pipeline: NHTSA data in, a layered warehouse, machine-learning detection, and AI briefs that are fact-checked."** Two paths: the **write path** builds, the **read path** only reads, so users never interfere with the build.
3:20	SOURCES	Left: 8 public files, about **6.8M rows**, republished daily. Landing zone: **ETag check**, so only changed files download. **One input, two warehouses.**
3:40	LOAD	Raw stored **exactly as published**, with lineage; bad rows are kept, not dropped. The **quality gate** stops the run before dbt if a critical check fails.
4:00	DBT	Centre box: **one dbt project, two targets.** Seeds → staging → star schema → 64 tests, on DuckDB and Snowflake, **verified identical**.
4:20	DUCKDB	Top lane keeps **every layer plus ML**: embeddings → clusters per vehicle line → a **causal** spike test → recall matching → alerts → back-test. Read path: the dashboard, MCP server and agents through one read-only query layer.
4:50	SNOWFLAKE	Bottom lane: files staged and **COPY**-loaded, then raw → marts built by the same dbt. Service user with key-pair login, smallest warehouse, auto-suspend: **under one credit**. ML moves here next.
5:10	RESULTS	Airflow runs it daily at 06:00. Back-test: **153 of 440 detectable recalls flagged early (35%, target 50%)**, a median **9 months** before the recall and **11 months** before NHTSA opens an investigation.

System architecture · about 45 seconds

5:30	COMPONENTS	**"Same system, seen as components."** Ingest → Transform → Detect fill the warehouse; one query service feeds three consumers.
5:45	EXPLAIN	The agent loop: **signal** → **retrieval** → **drafting** → **verifier**. Every claim must cite a real complaint, recall, bulletin or investigation ID; the verifier rejects the rest and redrafts if too many fail. **Only verified claims are published.**
6:05	CLOSE	Delivered through the dashboard, the VIN check and an MCP server any AI assistant can call. **"Statistics detect, AI explains, and a verifier checks every claim."**

If asked. Why two databases? DuckDB for local dev and demos, Snowflake for the shared cloud; one dbt project, proven identical. · **No vector DB?** ~350K vectors; SQL similarity search is enough. · **Why 35%?** Matching misses and noisy clusters; fixes: rolling re-clustering, better matching. · **Hallucinations?** The verifier fails closed.